

AUTOMATIC CHORD RECOGNITION WITH GENERATIVE MODELS

Anonymous Authors
Anonymous Affiliations
anonymous@ismir.net

ABSTRACT

Automatic Chord Recognition (ACR) has been a problem of interest to the music information retrieval community for over a decade, yet performance has not meaningfully improved in that time and the problem remains unsolved. A key limiting factor is the lack of labelled data. In this work, we investigate the use of generative music models to address this. We extract embeddings from modern generative models for use as input features to an ACR classifier. We also create new training data using a chord-condition generative model. Finally, we rethink how the model predicts chords in time by incorporating beat estimation. While performance improvements are small, synthetic data shows promise in correcting model failures, and oracle beat estimation yields the strongest results seen in the field, paving a path forward.

1. INTRODUCTION

Chord annotations allow music to be easily shared, performed, improvised and analysed. Creating high-quality chord annotations requires a trained musician and can be highly subjective [1]. Thus, ACR models are a promising avenue for creating high-quality chord annotations at scale, allowing for better understanding, creation and learning of music. To this end, ACR has been tackled by deep learning methods since the influential work of [2]. They train a convolutional neural network (CNN) to classify pitch spectra into chord labels. Progress was made by combining CNNs with recurrent neural networks (RNNs) [3–6] with a CNN performing feature extraction from a spectrogram and an RNN sharing information across frames. Such models are referred to as CRNNs. More recently, transformers have been applied in place of the RNN [7–13]. Despite increasingly complex models being employed for ACR, performance improvements have stagnated, and have barely improved in 10 years since the first CRNN trained by [3]. The ‘glass ceiling’ discussed by [14] is still present 10 years later.

The original work in [2] classified pitch spectra into 25 chords labels — all 12 keys of minor and major plus a no chord symbol. Most work since then has focused on a much larger vocabulary. The most common is the

170 chord vocabulary introduced by [6], which includes 14 chord qualities, plus a no chord and unknown chord symbols. Such a large chord vocabulary creates a long tail of rare chord labels. For example, major chords are very common, while minmaj7 chords are extremely rare in our dataset. Detailed chord annotations can also be highly subjective [1]. For example, annotators may disagree on the presence of a chord extension. Such subjectivity may make it harder for a model to learn a consistent mapping from audio to chord labels, and brings into question the upper bound of performance for ACR models.

In this work, we propose the lack of performance improvement in ACR is largely due to the limitation imposed by the size of the dataset. The most common dataset used for ACR — and the dataset used in this work and other works over the last decade — contains around 1200 annotated pop songs. By contrast, Jukebox [15], a generative music model published over 5 years ago, was trained on 1.2 million songs. It is infeasible to obtain an annotated dataset of any comparable size for ACR. Instead, we must find ways of leveraging self-supervised methods, models from other musical domains or generative models to learn from a larger source of data, without more chord labels. Here, we propose three methods: (1) extracting *generative features* from MusicGen [16] as input to a CRNN (2) using a chord-conditioned generative model to create synthetic data to train on and (3) using a beat estimation model to inform the model of when chord changes are likely to occur. We perform our experiments by reimplementing the CRNN from [6] and ablating on various common performance improvements. This includes output smoothing, structured decomposition of the chord vocabulary, and pitch augmentation. We then apply the three methods described above and find that none meaningfully improve performance. However, training on synthetic data does improve performance marginally, and changes the types of errors made by the model. We also find that with oracle beat estimation, performance improves significantly. All this suggests that better synthetic datasets and beat estimation models could be a promising avenue for improving ACR performance and breaking through the current ‘glass ceiling’. All code and a more detailed manuscript containing all experimental results are available on GitHub.¹ Data can be made available upon request.²



¹ redacted for anonymity

² redacted for anonymity

2. RELATED WORK

The field of ACR has seen considerable research since the seminal work of [17] in 1999. Below, we provide a brief overview of the field including the models that are commonly used and the methods used to improve performance. We then highlight the use of generative models and the use of frames and beats in ACR and other music information retrieval tasks in order to motivate the methods we apply in this work.

Models: Increasingly complex models have been applied, following the trends in other fields. This started with CNNs [2], then these were combined with recurrent neural networks (RNNs) [3–6] with a CNN performing feature extraction from a spectrogram and an RNN sharing information across frames. We refer to these models as CRNNs. More recently, transformers have been applied in place of the RNN [7–13]. [14] discuss the ‘glass ceiling’ in ACR performance that is still largely present today. The ‘BTC’ model introduced by [11] has become a common baseline despite the lack of performance improvement in the original publication. The more recent ‘Chordformer’ [13] shows a modest improvement of up to 2%. Instead, for reasons of computational constraint and the wish to run many experiments, we choose to use the CRNN from [6] as our baseline model. This remains competitive with state-of-the-art and we hope that performance improvements on this model are representative of the possible improvement with more complex models.

Improvement Methods: *Output smoothing* — Frame-by-frame chord prediction can create unstable outputs with many short chord transitions. To this end, the use of hidden mark models (HMMs) [18] and conditional random fields (CRFs) [5] have been used as a post-processing step for output smoothing. *Pitch Augmentation* — First introduced by [6], models are trained with pitch alterations as a form of data augmentation to encourage root-invariance in the model and to increase the model’s ability to generalise. *Structured Decomposition* — It is now common to decompose the chord vocabulary into root, quality and extensions and to have the model predict these separately [5, 6]. Such decomposition is thought to increase performance by around 1%, with greater gains at recognising the root, third and seventh. *Weighting the loss* — To address the long tail of the chord label distribution, it is common to increase the weight of the loss on rare chord classes [5]. We perform extensive ablations on four of these improvement methods: output smoothing, weighting the loss function, structured decomposition, pitch augmentation.

Other methods have been attempted but are not explored in this work as they are less common or computationally too expensive for us. For example, some have suggested embracing the subjective nature of chord annotation [14, 19]. This has been attempted by [12] who use label-smoothing informed by a consonance-based distance metric between chord labels.

Generative Models: Generative models have been transformative in other domains such as natural language processing and computer vision. In such domains, large

generative models have become the dominant method for performing all classification and generation tasks. It is common for classification to be performed on embeddings produced by large, pre-trained, generative models, and to use generative models to create large synthetic datasets for further training. In music, generative models such as Jukebox [15] and MusicLM [20] have transformed the ability to quickly create some styles of music. They have been employed for music transcription of melody and chords [7] but extensive analysis for ACR using the dominant dataset is missing. We propose to follow the example set by other domains and use generative models for ACR in two ways: (1) using features extracted from a generative model as input to a CRNN and (2) using a chord-conditioned generative model to create synthetic data to train on. We use generative features extracted from MusicGen [16] and use a MusiConGen [21], a chord-conditioned generative model, to generate synthetic training data for ACR.

Frames and Beats: Chords exist in time. ACR is most commonly performed on a constant-Q transform (CQT) of the audio signal, with the frame length determining the temporal resolution. A variety of frame lengths have been used to perform ACR, ranging from 512 [5] to 4096 [6]. By contrast, [7] perform melody and chord transcription on a Mel-spectrogram [22] by resampling generative features onto a frame length that is song-dependent and beat-aware. They use *madmom* [23] for beat estimation and resample their features to a grid of 1/16th notes. We propose to use beat detection models from [23] to first identify beats in the song, and to investigate the relationship between varying the granularity of the beat grid and the performance of the ACR model. We also perform ablations on the CQT frame length and the type of spectrogram used as input to ensure that current best practices are reasonable.

3. METHOD

Data: We train and evaluate on a dataset we refer to as *pop*. This is comprised of annotations from four sources — (1) *McGill Billboard* [24]: songs randomly selected from the Billboard ‘Hot 100’ Chart between 1958 and 1991, (2) *Iso-phonics* [25]: songs from albums by The Beatles, Carole King and Zweieck, (3) *RWC-Pop* [26]: pop songs with annotations available³, and (4) *USPop* [27] songs chosen for popularity. Annotations and audio were kindly provided by anonymous members of the ACR community. Much data integrity and preprocessing work was conducted to ensure the dataset was clean and aligned. 4 songs were not in the same tuning as the annotations, or had incorrect annotations and were removed. In total, this leaves 1,213 songs.

We split the data into 60/20/20 train/validation/test splits in a song-wise manner. We report results on the validation set for all experiments, except the final results where we train on the combined train and validation set and report on the test set, as is best practice in machine learning. While this is not the common practice in ACR, we feel that a sizeable validation set is important to obtain

³ <https://github.com/tmc323/Chord-Annotations>

199 good estimates of the generalisation error, and that cross- 257
200 fold validation may distort results in the literature [28]. 258

201 All audio signals are converted into a CQT using a hop 259
202 length of 4096 and a sampling rate of 44,100Hz using 260
203 `librosa` [29]. Different previous works have used dif- 261
204 ferent hop lengths. We thus test different hop lengths and 262
205 find that 4096 attains among the best performances while 263
206 maintaining low computational cost. 264

207 **Evaluation:** We use the standard weighted chord sym- 265
208 bol recall (WCSR) metric for evaluation. However, to fa- 266
209 cilitate fair comparisons over differing frame lengths and 267
210 beats, we define this using an integral over time, rather than 268
211 discrete sum over frames as is standard. A formal defini- 269
212 tion is omitted but can be found in the full manuscript and 270
213 code. WCSR is then the percentage of time that the pre- 271
214 dicted chord label matches the ground truth chord label, 272
215 where correctness can be measured by a variety of met- 273
216 rics. Such metrics include exact matching (or just ‘acc’), 274
217 `root`, `third` and `seventh` as well as `mirex` which re- 275
218 quires that two chords match in at least three pitches. This 276
219 measure is the most forgiving, as it allows for missing up- 277
220 per extension, incorrect inversions or `maj/min7` being con- 278
221 fused with the relative `min/maj` respectively. All metrics
222 are implemented in the `mir_eval` library [30]. We also 279
223 report the mean class-wise WCSR, which is the average of 280
224 the WCSR for each chord class. This provides a measure
225 of performance on the long tail of the chord distribution. 281

226 **CRNN:** All experiments are performed using a CRNN, 282
227 as described by [6]. The model consists of a CNN fea- 283
228 ture extractor followed by a bidirectional GRU. We ablate 284
229 on the model architecture by comparison to a CNN-only 285
230 model, and varying the width and depth of the CNN and 286
231 GRU with random search. We find that performance is 287
232 not sensitive to hyperparameters within reasonable bounds 288
233 and that those proposed by [6] are among the best choices. 289
234 Thus, we stick with these choices for our baseline model. 290
235 We use this CRNN as our baseline model for all experi- 291
236 ments. 292

237 We do not train a transformer model, both due to 293
238 computational constraints and because the original ‘BTC’ 294
239 model is outperformed by a CRNN [11]. Further, we 295
240 ran experiments increasing numbers of parameters in the 296
241 CRNN which suggested to us that model complexity is not 297
242 the bottleneck. Finally, we seek to run many experiments 298
243 to find sources of improvement, not to train the single best 299
244 model. A lighter CRNN is more suited to this task. 300

245 **Training:** A training epoch consists of randomly sam- 301
246 pling a patch of audio from each song in the training set. 302
247 The length of this sample is kept as a hyperparameter, set 303
248 to 10 seconds for the majority of experiments. This length 304
249 of patch was also tested and was found to have little effect 305
250 on performance. For evaluation, the entire song is used 306
251 as performance was found to be marginally better. When 307
252 validating mid-way through training, songs are split into 308
253 patches of the same length as the training patches to save 309
254 on computation time. Samples in a batch are padded to 310
255 the maximum length of a sample in the batch and padded 311
256 frames are ignored for loss and metric calculation.

Experiments are run on GTX Titan Xs (12GB VRAM), RTX A6000s (48GB VRAM) and CPUs when GPUs were unavailable. All models are trained with the Adam optimiser [31] with a learning rate of 0.001 and cosine scheduling. Models are trained to minimise the cross entropy loss between the predicted chord and the true chord distribution. We use a batch size of 64 and train for 150 epochs. Validation part-way through training is conducted every 5 epochs. The model is saved whenever the validation loss improves. Each training run takes approximately 30 minutes of GPU time or 1 hour 30 minutes of CPU time, though this varies significantly across experiments.

We apply four standard improvements to the baseline CRNN. We smooth outputs with an HMM, reducing predicted transitions from 168 to 102 per song to match the ground truth rate of 104. We weight the loss by inverse class frequency to improve performance on rare chord qualities, like in [5]. We decompose the output into root, third and seventh predictions following [6, 12, 13]. Finally, we perform pitch augmentation by shifting the CQT by a random number of semitones with probability $p = 0.9$, effectively multiplying the training data by a factor of 12.

4. EXPERIMENTS

4.1 Generative Features

[7] use generative features extracted from Jukebox [15] to improve performance for melody transcription. They also produce a chord transcription model using the same methodology but do not report results. We test generative features using MusicGen [16] as a feature extractor. This was for several reasons. MusicGen is a newer model. It has several different sizes of models that can be tested to ablate on the complexity of the model. It has a fine-tuned variant called MusiConGen [21] which is used for synthetic data generation in Section 4.2. All model weights are available on the HuggingFace Hub.⁴ Finally, its training data was properly licensed, unlike Jukebox. We leave the results using Jukebox features and other models for future work.

Overlapping 5 second chunks of audio were fed through MusicGen in a batched fashion. This first requires passing the audio through the pre-trained Encodec audio tokeniser [32]. These are then fed through the language model. We take the output logits as the representation for each frame. The model outputs logits in four ‘codebooks’, each 2048-dimensional vectors, intended to represent different granularities of detail in the audio. Audio segments are overlapped such that every frame has context from both directions. The multiple representations for each frame are averaged. Finally, these representations are upsampled. The model operates at a frame rate of 50Hz. To compute a representation with the same frame length as the CQT, we take the mean over the frames outputted by the model closest to the centre of the CQT frame. In case averaging over frames dampened the signal, we also tried linearly interpolating between the two closest frames outputted by the model. This was empirically found to perform slightly

⁴<https://huggingface.co/docs/hub/en/index>

features	accuracy	root	third	seventh	mirex
gen-CQT	61.0	80.3	77.0	63.3	78.4
gen	58.7	77.6	74.3	60.9	77.5
CQT	61.0	79.8	76.8	63.2	79.3

Table 1. Comparison of CQT and generative features with a concatenation of the two, denoted as gen-CQT. The concatenation performs the best on most metrics but not by enough to claim to be meaningfully better.

worse. Due to computational constraints, pitch augmentation was removed for generative feature experiments.

As the representations are 2048-dimensional, it is computationally infeasible to feed this directly into the GRU. Instead, using fully connected layers, we project these vectors down to a power of 2, from 16 to 1024. The best representation is with 64 dimensions, although results show no clear trend.

We test different variants of MusicGen, including musicgen-large (3.3B parameters), musicgen-small (330M), musicgen-melody (1.5B) and MusiConGen (1.5B). We also test different reductions of the four ‘codebooks’ outputted by the language model. This includes taking each codebook on its own, the element-wise mean across codebooks and the concatenation of all four. For further details on codebooks, see the work of [16]. The best model was found to be musicgen-large and the best reduction to average over the four codebooks. However, results were close and are omitted for lack of interest.

To test whether or not these features help when compared with a CQT, we test with the CQT only, generative features only and a concatenation of the two. The results are shown in Table 1. Although the generative features perform worse than the CQT, they contain information useful for chord recognition with an accuracy of 58.7%. Performance remains largely the same when the CQT and generative features are used together. This experiment was run multiple times, with similar results each time. There is no clear evidence that the generative features from MusicGen provide any benefit over just using the CQT.

This conclusion is surprising as [7] claim that generative features are better than hand-crafted features for the related task of melody recognition. However, they only compare to mel-spectrograms, which may not perform as well as CQTs, as they certainly do not for chord recognition. Observations here cast doubt on how well their claims generalise to chord recognition. Features extracted from other generative models such as Jukebox [15] or MusicLM [20] may perform better as MusicGen was trained with only licensed music, severely limiting the portion of the full distribution it learnt structure about. The comparison is left for future work. Another explanation is that [7] use Mel-spectrogram features as baselines. We perform a further experiment with results showing that CQTs improve over Mel-spectrograms by around 10% for ACR. Thus, the CQT baseline may be stronger in ACR than Mel-spectrograms for melody transcription.

4.2 Synthetic Data Generation

Given the success of pitch augmentation for ACR, it is sensible to look for other sources of data. Further augmentation is possible by adding noise and time-stretching. However, these do not provide new harmonic structures or create new instances of rare chords. Instead, we look to generate new data, taking into account our understanding of harmonic structure. Generation would be possible through automatic arrangement, production and synthesis software. However, this is a complex task requiring a lot of human input and is unlikely to produce a sufficient variety of timbres, instrumentation and arrangement. Instead, we use a recent chord-conditioned generative model called MusiConGen [21]. We use this over CocoMulla [33] as the method of chord-conditioning has a more straightforward interface, doesn’t require reference audio and the authors claim that it adheres more closely to its chord conditions. Indeed, they feed the outputted audio through BTC [11] and find a `triads` score of 71% using the chord conditions as the ground truth. While far from perfect, it suggests the model can generate audio that mostly adheres to the chord conditions.

We generate 1210 songs, each 30 seconds long, to mimic the size of the *pop* dataset. We refer to this dataset as *synth*. This is split into a train/validation/test split in the same fashion as for the *pop* dataset. Generating a larger order of magnitude of audio would require more computational resources. While the model supports auto-regressive generation for longer audio, its outputs become incoherent using the provided generation functions. It also sometimes produces incoherent output with 30 seconds of generation, but this was much less common.

To generate a song, we sample a BPM from $\mathcal{N}(117, 27)$ and a song description from a set of 20 generated by ChatGPT. The model does not output melodic vocals, owing to the lack of vocal music in the pre-training and fine-tuning data. Finally, we generate a jazz chord progression using the theory of functional harmony [34]. The process generates a very different chord distribution than the one in *pop* with many more instances of upper extensions and rare qualities. This is intended to provide the model with many more instances of rare chords.

To help correct for the distribution shift between synthetic training data and the *pop* train split, we estimate the empirical class probabilities in each domain— $P_{\text{train}}(y)$ and $P_{\text{pop}}(y)$ —and rescale the model’s logits by the ratio

$$r(y) = \frac{P_{\text{pop}}(y)}{P_{\text{train}}(y)}. \quad (1)$$

In order for calibration to be root invariant, we take the mean ratio over chords that share the same quality, and a single calibration factor r_q is applied to every chord with that quality. Note that the model’s outputs are in logits so the log ratio is added in implementation.

We manually inspect outputs from MusiConGen. In general, the outputs are good. They consistently stick to the specified BPM and usually stick to the chord conditions. Outputs are occasionally musically strange with jar-

ring drum transients and unrealistic chord transitions. They also did not have an enormous variety in timbre and instrumentation. Nonetheless, most examples have sensible annotations that one would expect a human to be able to annotate well.

We empirically compare a model trained on only *pop*, trained on only synthetic data and trained on both. While the latter results in more training data per epoch, convergence is always reached, so these are fair comparisons. We test the models on the *pop* and synthetic data validation splits. Given the increased instances of rare chords in the synthetic data, we remove the weighting on the loss function for the models trained on synthetic data.

Table 2 shows the results. The model trained on both datasets performs very similarly to the model only trained on *pop*. That is, the model has not overfit to the synthetic data. However, it has also not improved performance. Training on synthetic data drastically improves the accuracy on the *synth* validation split. This is likely due to the unique chord distribution of the generated jazz progressions and the consistent instrumentation. The model trained on both datasets performs the best, with an accuracy of 51.0%.

train set	<i>pop</i> acc	<i>pop</i> third	<i>pop</i> seventh	<i>synth</i> acc
<i>pop</i>	62.1	77.7	64.4	24.2
<i>synth</i>	48.6	64.4	50.6	44.8
both	62.1	77.9	64.6	51.0

Table 2. Results for models trained on *pop*, *synth* and both together.

division	acc	root	third	seventh	mirex
1/4	62.3	81.3	78.3	64.5	79.6
1/2	62.8	81.5	78.6	65.1	79.9
1	62.3	81.3	78.1	64.6	80.0
2	56.4	74.7	71.6	58.5	73.4
none	62.5	81.7	78.2	64.7	80.2
oracle	61.1	79.6	76.1	63.4	90.4

Table 3. Results for different beat divisions. The beat division of none refers to a frame-wise *CRNN* with HMM smoothing applied, ‘1’ refers to the beats from madmom and other numbers correspond to the relevant subdivision/grouping, and ‘oracle’ refers to the correct chord intervals.

For further insight, the difference in confusion matrices over chord qualities is plotted in Figure 1. Recall increases for some rare qualities and decreases for others. Notably, recall on *min7* improves by 13% and the model is much better at predicting the third in dominant and *min7* chords. This is likely why the third and seventh accuracies increase slightly on the *pop* validation set. Training on synthetic data discourages the model from predicting X chords, which are not present in the synthetic data. Also,

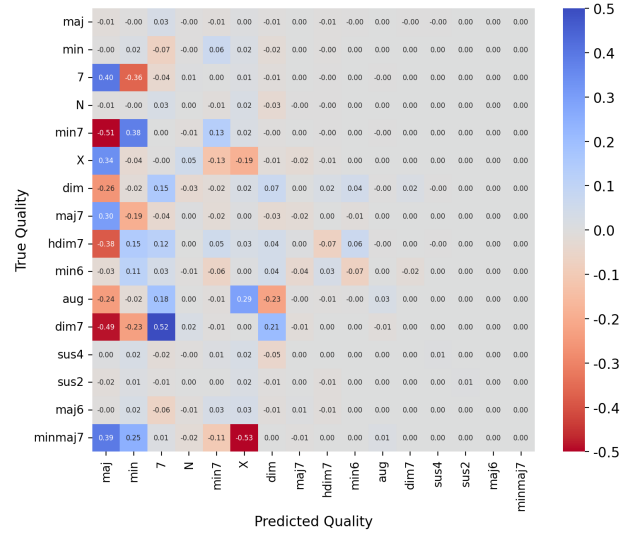


Figure 1. Difference in normalised confusion matrix of the model trained on synthetic data and *pop* data versus just *pop* data. Training on synthetic data discourages the model from predicting X chords, which are not present in the synthetic data. Recall on *min7* and *dim* qualities increases. However, recall on *hdim7* and *min6* worsens. In general, the model predicts *maj* less often for rarer qualities.

many predictions on 7 qualities change from *min* to *maj*, which is a closer chord. Something similar happens for *min7* chords. Thus, training on synthetic data can alter the kinds of mistakes the model is making for the better.

A problem is that the chord-conditioned model was trained using an ACR model [11] which is fundamentally limited in its representations of harmonic structure. These limitations are the same ones we are trying to break through using synthetic data. This is akin to the problem of model collapse when recursively training on synthetic data [35].

While performance does not meaningfully improve on *pop*, the lack of overfitting and the gain on the synthetic data provide hope that synthetic data could prove a useful tool for ACR with further work and improved generative models. Furthermore, producing hand-annotated data is error-prone and subject to human interpretation. Synthetic data may produce non-identifiable data points but is consistent and error-free. Poor performance on synthetic data can only be explained by failures of the model or indeterminacy of the generated samples, not incorrect labels.

4.3 Beat Synchronisation

Chords exist in time. Musicians interpret chords in songs as lasting for a certain number of beats, not a fixed length in time. In its current form, the model outputs frame-wise predictions. While these could be stitched together to produce a predicted chord progression or beat-wise predictions could be made as a post-processing step, we decide to implement a model that outputs beat-wise predictions directly. This allows the model to use information from

	acc	root	third	seventh	mirex	acc _{class}
<i>CRNN</i>	62.8	80.9	78.3	64.6	80.6	18.6
+ gen features	62.7	80.4	77.9	64.6	80.6	19.5
+ pitch shift	63.8	82.4	79.4	65.6	80.3	19.7
+ synthetic data	64.4	82.2	79.9	66.3	81.5	18.3
+ beats	63.7	82.4	79.7	65.6	80.9	19.7
+ oracle beats	65.8	84.5	81.7	67.6	88.7	21.2

Table 4. Final results from various experimental setups on the test set. Best results excluding the oracle are bolded.

the entire duration of the beat to make its prediction. Following the methodology of [7], we first detect beats using *madmom* [23]. This returns a list of time steps where beats have been detected. We first verify that these beats are plausible by computing the maximum accuracy a model could attain if predicting chords at the whole beat level. This is done by iterating over each beat interval and assigning the chord with maximum overlap with the ground truth. This yields an accuracy of 97.1%, thus we are satisfied that the estimated beats are accurate enough to be used.

To calculate features for a beat interval, we average all CQT features whose centre is contained within the interval. The CQT is calculated using a hop length of 1024. The shorter hop length is used to minimise the effect of CQT frames with partial overlap with two beat intervals and ensures that each beat has many CQT frames associated with it. These representations have the added benefit of decreased computational cost as beats have a lower frequency than frames.

We test the model with different divisions and groupings of beats. This tests the assumption that whole beats are fine-grained enough for the model to make good predictions. We include tests where beat intervals are subdivided into two or four, or beat intervals are joined into groups of two. We refer to this as the *beat division*. We also test *oracle* beat divisions where the true chord transitions are taken as the beats. This is not a fair comparison as the model should not have access to true transition timings. However, it does provide an idea of how the model would fare if it could perfectly predict chord transitions. HMM smoothing is removed for beat-wise models as it is no longer necessary.

Results are shown in Table 3. Using beat-wise predictions does not affect performance compared to frame-wise predictions. The model performs just as well with a beat division of 1 as with a beat division of 1/2 or 1/4. The model performs worse with a beat division of 2, though it only loses 6% accuracy. This suggests that most chord transitions occur at frequencies lower than the beats produced by *madmom*.

The *oracle* model performs slightly worse on accuracy metrics but attains a very high *mirex* score of 90.4%, the highest seen in the literature. Data from the CQT producing a *mirex* of 90.4% provides hope for significant improvements in the field. Why the model performs worse on accuracy metrics is not clear. It may be because aver-

aged features from longer time periods provide better information as to the pitch classes present but dampen signal regarding the root note. Indeed, the mean chord duration is 1.68 seconds while a CQT frame is 0.093 seconds. Why this effect is not observed when predicting at the beat level is also not clear.

4.4 Final Results

For final results, we retrain select models on the combined training and validation splits and test on the held-out test split. This is an 80/20% train/test split. We consider the *CRNN* with a weighted and structured loss and HMM smoothing, concatenating generative features with CQTs, pitch augmentation, beat-wise predictions, ‘oracle’ beat-wise predictions and training on synthetic data.

Results are shown in Table 4. Observations are largely similar to those found previously. Generative features do not help and synthetic data improves performance by an additional 0.6%. This alone is not a clear enough signal that training on synthetic data is better, but it provides hope for further work. Finally, beat-wise predictions maintain the same performance. The ‘oracle’ model achieves the highest performance across all metrics. This is still a significant result and suggests that there is hope for breaking through the ‘glass ceiling’ through better beat detection. The model’s performance on the long tail remains poor.

5. CONCLUSION

In this paper, we presented three new methods of leveraging external models to try to improve performance on ACR. Using generative features, training on synthetic data and performing beat estimation did not significantly improve performance. However, training on synthetic data did improve performance a little and massively on synthetic data. Further, it changed model behaviour in meaningfully positive ways. Further, oracle beat estimation produced the greatest *mirex* results seen in the field, providing hope that the ‘glass ceiling’ may yet yield. Future work could use a greater variety of generative models trained on more data, attempt to create a greater quantity and higher quality of synthetic data or develop better methods of beat estimation that is pitch-aware to bring performance closer to oracle beat-tracking.

6. ETHICS STATEMENT

This work is severely limited by the dataset. Songs mostly conform to modern Western standards in pop music which is far from a holistic theory of music. Our work contributes to the ongoing globalisation of the dominant Western culture in music and beyond.

Machine learning is having an increasing impact on the environment through the data centers and energy required to train them, as well as all of the materials required to build advanced computers. We were limited by computational constraints in this work and so did not undertake any large scale experiments, nor massively increase demand for ever-expanding data centers.

Training on copyrighted audio is a fraught and difficult issue to contend with, especially for generative models. The audio used in this work is kept private for this reason. However, we did not train any generative models. We hope that ACR represents a field in music information retrieval that is free of many of the worries for artists surrounding generative models. We did use generative models in this work, but restricted ourselves to generative models trained on licensed music.

7. REFERENCES

- [1] H. V. Kooops, W. B. de Haas, J. A. Burgoyne, J. Bransen, A. Kent-Muller, and A. Volk, "Annotator subjectivity in harmony annotations of popular music," *Journal of New Music Research*, vol. 48, no. 3, pp. 232–252, 2019.
- [2] E. J. Humphrey and J. P. Bello, "Rethinking automatic chord recognition with convolutional neural networks," in *2012 11th International Conference on Machine Learning and Applications*, vol. 2, 2012, pp. 357–362.
- [3] M. Mauch and S. Dixon, "Audio chord recognition with recurrent neural networks," in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, 2014.
- [4] Y. Wu, T. Carsault, and K. Yoshii, "Automatic chord estimation based on a frame-wise convolutional recurrent neural network with non-aligned annotations," in *2019 27th European Signal Processing Conference (EUSIPCO)*, 2019, pp. 1–5.
- [5] J. Jiang, K. Chen, W. Li, and G. G. Xia, "Large-vocabulary chord transcription via chord structure decomposition," in *International Society for Music Information Retrieval Conference*, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:208334209>
- [6] B. McFee and J. P. Bello, "Structured training for large-vocabulary chord recognition," in *International Society for Music Information Retrieval Conference*, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:3072806>
- [7] J. T. Chris Donahue and P. Liang, "Melody transcription via generative pre-training," 2022. [Online]. Available: <https://arxiv.org/abs/2212.01884>
- [8] T.-P. Chen and L. Su, "Harmony transformer: Incorporating chord segmentation into harmony recognition," in *International Society for Music Information Retrieval Conference*, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:208334896>
- [9] —, "Attend to chords: Improving harmonic analysis of symbolic music using transformer-based models," *Trans. Int. Soc. Music. Inf. Retr.*, vol. 4, pp. 1–13, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:232051159>
- [10] L. O. Rowe and G. Tzanetakis, "Curriculum learning for imbalanced classification in large vocabulary automatic chord recognition," in *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*, 2021, pp. 586–593. [Online]. Available: <https://archives.ismir.net/ismir2021/paper/000073.pdf>
- [11] J. Park, K. Choi, S. Jeon, D. Kim, and J. Park, "A bi-directional transformer for musical chord recognition," in *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, 2019, pp. 620–627. [Online]. Available: <https://archives.ismir.net/ismir2019/paper/000075.pdf>
- [12] A. Poltronieri, X. Serra, and M. Rocamora, "From discord to harmony: Decomposed consonance-based training for improved audio chord estimation," 2025. [Online]. Available: <https://arxiv.org/abs/2509.01588>
- [13] M. W. Akram, S. Dettori, V. Colla, and G. C. Buttazzo, "Chordformer: A conformer-based architecture for large-vocabulary audio chord recognition," 2025. [Online]. Available: <https://arxiv.org/abs/2502.11840>
- [14] E. J. Humphrey and J. P. Bello, "Four timely insights on automatic chord estimation," in *International Society for Music Information Retrieval Conference*, 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:18774190>
- [15] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," 2020. [Online]. Available: <https://arxiv.org/abs/2005.00341>
- [16] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Defossez, "Simple and controllable music generation," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 47 704–47 720. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/94b472a1842cd7c56dcb125fb2765fbd-Paper-Conference.pdf

- 669 [17] T. Fujishima, "Realtime chord recognition of mu- 720
670 sical sound: a system using common lisp mu- 721
671 sic," in *International Conference on Mathemat- 722
672 ics and Computing*, 1999. [Online]. Available: 723
673 <https://api.semanticscholar.org/CorpusID:38716842>
- 674 [18] J. Miller, K. O'Hanlon, and M. B. Sandler, "Improv- 725
675 ing balance in automatic chord recognition with ran- 726
676 dom forests," in *2022 30th European Signal Process- 727
677 ing Conference (EUSIPCO)*, 2022, pp. 244–248.
- 678 [19] J. Pauwels, K. O'Hanlon, E. Gómez, and M. B. 729
679 Sandler, "20 years of automatic chord recognition from 730
680 audio," in *International Society for Music Information 731
681 Retrieval Conference*, 2019. [Online]. Available: 732
682 <https://api.semanticscholar.org/CorpusID:208334309>
- 683 [20] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, 734
684 M. Verzett, A. Caillon, Q. Huang, A. Jansen, 735
685 A. Roberts, M. Tagliasacchi, M. Sharifi, N. Zeghidour, 736
686 and C. Frank, "Musiclm: Generating music from text," 737
687 *arXiv preprint arXiv:2301.11325*, 2023. [Online]. 738
688 Available: <https://arxiv.org/abs/2301.11325>
- 689 [21] Y.-H. Lan, W.-Y. Hsiao, H.-C. Cheng, and Y.-H. 741
690 Yang, "Musicongen: Rhythm and chord control 742
691 for transformer-based text-to-music generation," in 743
692 *Proceedings of the 25th International Society for 744
693 Music Information Retrieval Conference (ISMIR)*, San 745
694 Francisco, United States, 2024. [Online]. Available: 746
695 <https://arxiv.org/abs/2407.15060>
- 696 [22] S. S. Stevens, J. Volkman, and E. B. Newman, "A 747
697 scale for the measurement of the psychological mag- 748
698 nitude pitch," *The Journal of the Acoustical Society of 749
699 America*, vol. 8, no. 3, pp. 185–190, 1937.
- 700 [23] S. Böck, F. Korzeniowski, J. Schlüter, F. Krebs, and 750
701 G. Widmer, "madmom: a new Python Audio and Mu- 751
702 sic Signal Processing Library," in *Proceedings of the 752
703 24th ACM International Conference on Multimedia*, 753
704 Amsterdam, The Netherlands, 10 2016, pp. 1174– 754
705 1178.
- 706 [24] J. Burgoyne, J. Wild, and I. Fujinaga, "An expert 755
707 ground truth set for audio chord recognition and mu- 756
708 sic analysis." 01 2011, pp. 633–638.
- 709 [25] C. Cannam, C. Landone, and M. Sandler, "Omras2 757
710 metadata project," in *Proceedings of the 10th Interna- 758
711 tional Society for Music Information Retrieval Confer- 759
712 ence (ISMIR)*, 2009, pp. 309–310.
- 713 [26] M. Goto, H. Hashiguchi, T. Nishimura, and R. Oka, 760
714 "Rwc music database: Popular, classical, and jazz mu- 761
715 sic databases," 01 2002.
- 716 [27] A. Berenzweig, B. Logan, D. P. Ellis, and B. Whitman, 762
717 "A large-scale evaluation of acoustic and subjective 763
718 music-similarity measures," *Computer Music Journal*, 764
719 vol. 28, no. 2, pp. 63–76, 2004.
- 720 [28] S. Varma and R. Simon, "Bias in error estimation 765
721 when using cross-validation for model selection," *BMC 766
722 Bioinformatics*, vol. 7, no. 1, p. 91, 2006. [Online]. 767
723 Available: <https://doi.org/10.1186/1471-2105-7-91>
- 724 [29] B. McFee, C. Raffel, D. Liang, D. Ellis, M. Mcvica, 768
725 E. Battenberg, and O. Nieto, "librosa: Audio and music 769
726 signal analysis in python," 01 2015, pp. 18–24.
- 727 [30] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, 770
728 O. Nieto, D. Liang, and D. P. W. Ellis, "Mir_eval: 771
729 A transparent implementation of common mir met- 772
730 rics," in *International Society for Music Information 773
731 Retrieval Conference*, 2014. [Online]. Available: 774
732 <https://api.semanticscholar.org/CorpusID:17163281>
- 733 [31] D. P. Kingma and J. Ba, "Adam: A method 775
734 for stochastic optimization," in *3rd International 776
735 Conference on Learning Representations, ICLR 2015, 777
736 San Diego, CA, USA, May 7-9, 2015, Conference 778
737 Track Proceedings*, 2015. [Online]. Available: <http://arxiv.org/abs/1412.6980>
- 739 [32] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High 779
740 fidelity neural audio compression," 2022.
- 741 [33] L. Lin, G. Xia, J. Jiang, and Y. Zhang, "Content-based 780
742 controls for music large language modeling," 2023.
- 743 [34] M. Steedman, "A generative grammar for jazz chord 781
744 sequences," *Music Perception: An Interdisciplinary 782
745 Journal*, vol. 2, no. 1, pp. 52–77, 1984.
- 746 [35] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, 783
747 R. Anderson, and Y. Gal, "Ai models collapse when 784
748 trained on recursively generated data," *Nature*, vol. 785
749 631, pp. 755–759, 2024.